Citation Checker

MVP Roadmap — Reliability Engine with Thin Relevancy Layer

August 24 – October 31, 2026 · ~10 hrs/week · ~100 hours total · Python · CLI-first · $0 tooling

What this builds

A command-line tool that takes an LLM answer, pulls out its citations, and reports two things for each one: how reliable the cited source is, and whether the page holds up against the claim it was attached to. Output is a band plus the signals behind it — not a single score.

Scope decisions already made

- **CLI, not a website.** A UI would eat 30+ hours and teach nothing about whether the engine works. Web layer is v2; a `--json` flag in Phase 5 becomes its API.
- **Bands, not numeric scores.** Trusted / Mixed / Weak / Untrusted, each with the reasons that drove it. Numbers computed internally, never shown — no false precision.
- **Reliability is the bulk; relevancy stays thin but non-zero.** Full claim-support checking is a project of its own. The cheap deterministic slice still ships in v1 so the system has a testable vertical slice end to end.
- **Standing tier cut entirely.** Inbound citations and academic indexes need paid data or per-index integrations. Least essential, most expensive.
- **Free tooling throughout.** Every need below has multiple $0 options.

Two structural cautions

- **Reliability has no ground truth.** Your labels are your own judgment, so agreement means “matches what I would have said,” not “correct.” Fine — just know which one you are measuring.
- **Weeks 4–7 land in midterms.** That is the longest stretch and the riskiest phase. If anything slips it will be there, which is why Phase 3 carries an explicit cut line.

Timeline at a glance

Phase	Focus	Weeks	Hrs
0	Decisions & skeleton	1	~5
1	Fetch layer	1–2	~15
2	Ground truth — 50 labeled citations	3	~8
3	Reliability engine	4–7	~35
4	Thin relevancy	8	~15
5	Verdict, output, buffer	9–10	~20

Every phase ends with something that runs. There is no stretch where you are building for three weeks with nothing to show.

Phase 0 — Decisions & Skeleton

Week 1
~5 hrs

Nail down what you are building before you build it. Cheap now, expensive to retrofit.

- Repo, venv, dependency management, `.env` for API keys
- Pick the subject area for test citations; write down what counts as in-scope
- Define the core data model — this ripples through everything after:
  - `Citation` — url, claim text, char offsets, source answer
  - `PageFetch` — html, extracted text, status, failure reason
  - `Score` — band, internal numeric, signal list
  - `Verdict` — the combined output
- Name the four support outcomes and the four reliability bands; put them in a constants file
- CLI entry point accepting pasted text or a file path
- SQLite cache schema: domain lookups keyed by domain, page fetches keyed by URL, results keyed by run

Exit: `python check.py answer.txt` *prints "Found 3 citations" with the URLs.*

> **Recommended subject area:** recent tech and software news. Claims verify quickly, source quality varies widely (strong outlets, SEO farms, forums, vendor blogs), and paywalls are less common than in news or academia. Avoid health and law — highest stakes, slowest to verify by hand.

Phase 1 — Fetch Layer

Weeks 1–2
~15 hrs

Unglamorous and unavoidable. This is where surprise hours go.

- Citation parser — markdown links, bare URLs, `[1]`-style footnotes with a reference block
- URL normalization — strip tracking params, resolve redirects, canonicalize
- Basic fetch with sane timeouts and a real user-agent
- Headless-browser fallback for JS-rendered pages
- PDF text extraction
- Main content extraction — strip nav, ads, boilerplate
- Cache every fetch to SQLite so re-runs over your test set are instant

Failure taxonomy — each gets its own code

dead (4xx/5xx) · timeout · paywall · bot-blocked · JS-required · PDF-unparseable · empty

Exit: *paste an answer, get clean text per citation or a specific failure code.*

> **Overrun risk.** This phase runs long more often than any other. If you pass 15 hours, ship it with the browser fallback triggered on demand only, and move on. Do not let fetching eat Phase 2.

Phase 2 — Ground Truth

Week 3
~8 hrs

The cheapest phase and the one that makes every phase after it measurable.

- Collect ~50 real citations by asking several LLMs questions in your subject area
- Build a small labeling script that shows you claim + page text and records your judgment
- Label each: reliability band, does the page support the claim, one-line reason
- Store as JSON or CSV in the repo
- Write `evaluate.py` — runs the tool over the labeled set and reports agreement

Exit: *a labeled file and a scoring script that currently reports near zero because nothing is built yet.*

> **Do not skip or postpone this.** Without labels you cannot tell whether Phase 6 work improved anything over Phase 5 work, and you will spend weeks tuning blind. It also surfaces extraction bugs fast, since those are obvious once you see them side by side.

Phase 3 — Reliability Engine

Weeks 4–7
~35 hrs

The bulk of the build, and where the schedule risk lives.

3a — Domain tier (~15 hrs)

- RDAP lookup → registration date, registrar, expiry
- Wayback history → first snapshot plus a **content continuity check** — registration gaps or an abrupt topic shift suggest an expired domain bought for its backlinks
- Certificate inspection — issuer, type, expiry, hostname match
- Domain reputation lookup against a free threat feed
- TLD prior at low weight — `.gov` / `.edu` / `.mil` are gated, cheap TLDs penalized
- Cache all of it per-domain; these signals barely change

3b — Accountability signals (~8 hrs)

- Detect: about/contact page, masthead, named staff, physical address, corrections policy, funding or ownership disclosure
- Heuristics first — link text, common paths, address regex. No model needed.
- Presence-based, cheap, and hard to fake at scale

3c — Content tier (~8 hrs)

- Author byline present and named
- Publication and last-updated dates

- Staleness measured *relative to claim volatility* — six months is fine for history, risky for security advisories
- Outbound citations — present, live, and the ratio of self-links to external
- Content type classification (news / opinion / press release / marketing / forum / academic / UGC) applied as a **weight modifier**, never as points. An opinion piece is not worse than a news article; it is a different thing judged differently.

3d — Scoring & calibration (~4 hrs)

- Combine signals into an internal numeric, map to a band
- Red-flag overrides — a threat-feed hit or a three-week-old domain caps the band regardless of what else scores well
- Run against your labels, tune weights

Exit: *URL in → band plus signal list out.*

> **Cut line:** if week 6 ends and 3a/3b are not solid, reduce 3c to author and date only. Do not push Phase 3 into Phase 4 — Phase 4 is what makes this project distinct from existing domain-rating tools.

Phase 4 — Thin Relevancy

Week 8
~15 hrs

The cheap slice of claim-checking. No NLI, no LLM, no decontextualization — that is v2.

- **Claim capture** — grab the sentence preceding each citation plus one sentence of context; store char offsets so you can highlight exactly what was checked
- **Structural checks** — dead URL, homepage-or-index instead of an article, couldn't-fetch. These are *outcomes*, not scores.
- **Deterministic matching only:**
  - Numbers and percentages — normalize “a third” against 33%, handle units
  - Dates across multiple formats
  - Quoted strings — exact match required when the LLM presents something as a quote

Outcomes

`contradicted` (number or date conflict) · `confirmed` (exact hit) · `not_checkable` (nothing deterministic to test) · plus the structural failures above

Exit: *a single run produces both dials.*

> **Why this stays in despite the scope pressure:** reliability alone has no ground truth and overlaps with tools that already exist. “The answer said 31%, the page says 47%” is checkable, provable, and the seed of the thing only your tool does.

Phase 5 — Verdict, Output & Buffer

Weeks 9–10
~20 hrs

- Combine the two dials into the four-quadrant verdict

- Terminal output — per-citation block with band, verdict, top three signals, plain-English reason
- `--json` flag for machine-readable output; this is your future web API
- Full run over the 50 labeled citations; record agreement
- Write down the failure cases you cannot fix — that is your v2 backlog, written from evidence
- README with setup and an honest limitations section
- **Buffer** for the two weeks you will lose

Stretch, only if genuinely ahead: a minimal web page over the JSON output.

The four-quadrant verdict

	Reliable source	Unreliable source
Claim supported	**Clean.** Nothing to flag.	**Faithful citation, bad source.** The LLM quoted correctly and still passed along something untrustworthy.
Claim not supported	**Misattribution.** Real source, does not say that. Expect this to be your most common hit.	**Junk throughout.**

The two middle cells are the product. Anyone can flag an obviously sketchy domain; almost nothing catches "this is a legitimate source that does not say what you were told it says."

Tooling options — research before committing

Every row is achievable at $0. Multiple options given deliberately so you can compare before locking in.

Need	Options
Domain age / registration	RDAP via the rdap.org bootstrap (official WHOIS successor, JSON, no key) · python-whois (raw WHOIS; parsing varies by registrar) · IP2WHOIS free tier · WhoisXML free tier · crt.sh first-certificate date as a proxy
Domain reputation	Google Safe Browsing Lookup API (free, key required) · URLhaus by abuse.ch (free, no key, bulk downloadable) · OpenPhish community feed · Spamhaus DBL over DNS (free at low volume) · VirusTotal free tier · Cloudflare Radar ranking API
Certificate inspection	ssl + socket from the standard library · cryptography for parsing detail · crt.sh Certificate Transparency logs (issuance history) · SSL Labs API (thorough, slow, heavily rate-limited)
Age continuity / history	Wayback CDX API (free, no key — best option) · Wayback availability API (simpler, less data) · crt.sh issuance gaps · Common Crawl index (free but heavy)
Fetching	requests · httpx (async — noticeably faster across 50 URLs) · Playwright for JS-rendered pages · Selenium
Content extraction	trafilatura (recommended — also pulls metadata) · readability-lxml · newspaper3k · boilerpy3 · jusText

Need	Options
Structured metadata	extruct for JSON-LD and microdata · OpenGraph tag parsing · htmldate for publication dates
PDF text	pdfplumber · PyMuPDF · pypdf · pdfminer.six
Storage	sqlite3 from the standard library · SQLModel · peewee
CLI & output	argparse from the standard library · typer · click · rich for readable terminal blocks

> **Try JSON-LD first.** Many article pages publish author, publish date, updated date, and article type as structured data. That is a large share of the Phase 3c content tier available for free, with no heuristics — check for it before writing byline regexes.

---

v2 backlog — deliberately deferred

Each of these was cut for a reason, not forgotten.

- **NLI verdict layer** — small entailment models (DeBERTa-family, MiniCheck, AlignScore, SummaC) give supports / contradicts / neither without a generative model. Fast, cheap, deterministic, different failure modes from an LLM.
- **Claim decontextualization and compound splitting** — resolving “this study” and separating “founded in 2003 and employs 400” into two checkable facts
- **LLM fallback** for low-confidence cases and for writing human-readable explanations — use a *different* model from the one being audited
- **Answer-level analysis** — independent corroboration, catching five citations that all trace back to one press release
- **Standing tier** — inbound citations, academic index presence, retraction records
- **Tone and framing analysis** — loaded language, headline/body mismatch, one-sidedness, sponsored markers, affiliate saturation. Note this measures *presentation*, not accuracy; name it accordingly.
- **AI-generation likelihood, ad density, SEO-farm markers**
- **Structured API input** — some providers return exact character spans per citation, which removes the scope-guessing problem entirely
- **Web UI** over the JSON output

If the schedule slips

Decide now which piece dies rather than discovering it on October 28. In rough order of what to shed first:

- Content tier (3c) reduced to author and date only
- Accountability signals (3b) reduced to contact page and named author
- Deterministic matching narrowed to numbers only, dropping dates and quotes
- Structural relevancy checks alone — dead, homepage-not-article, couldn't-fetch

What should *not* be cut, in any scenario: the Phase 2 labeled set, and some form of relevancy output. Those are what make the project measurable and what make it yours.

Roadmap drafted August 24, 2026 · target ship date October 31, 2026